

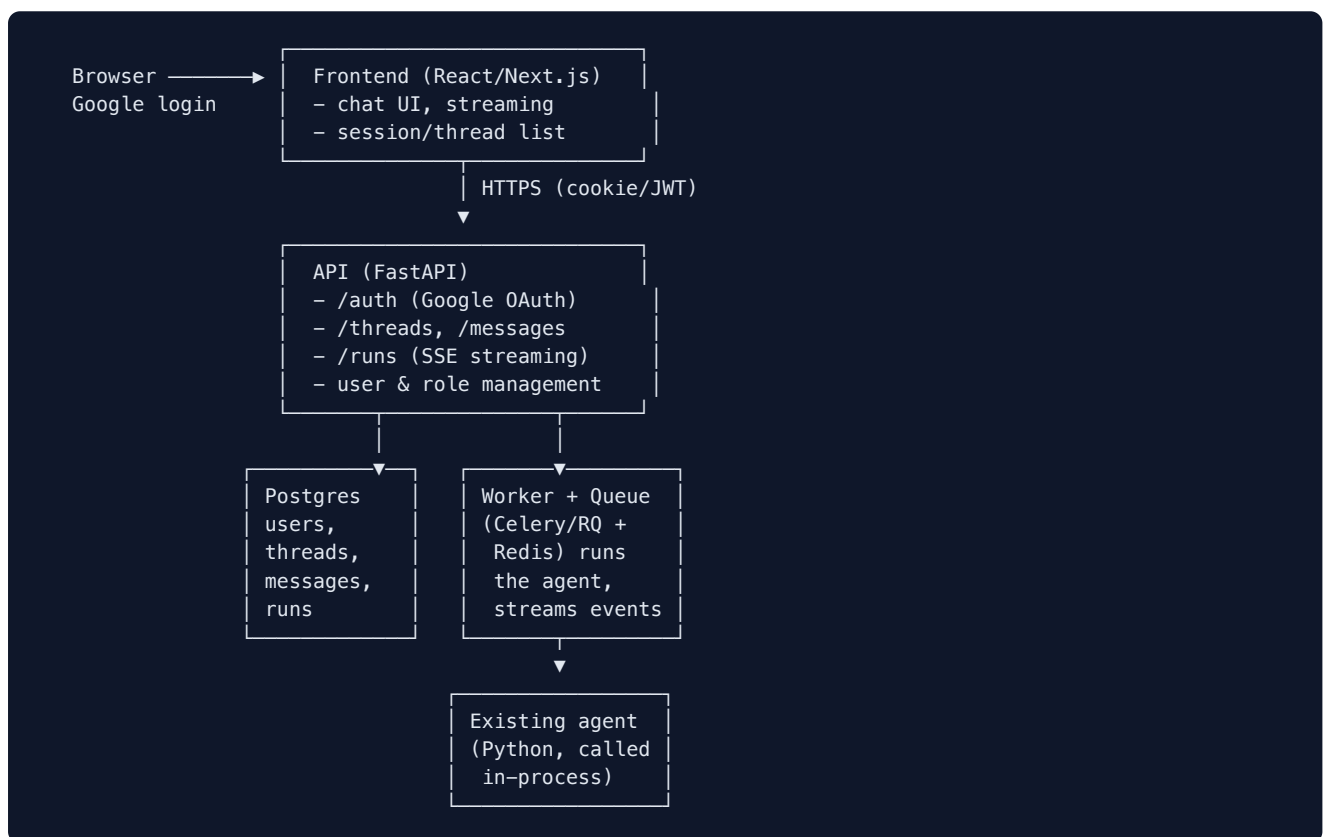
Build Plan: Web API + UI + Auth for the Agent

Goal: Turn the existing Python agent CLI into a web product with a REST/streaming API, a chat UI, user management, and Google SSO login.

Context / constraints (decided): - Existing agent is **Python**. - **Internal single-team** tool — Google SSO restricted to the company domain, lightweight roles (admin / member). - Agent runs are **long-running and streaming** — the API must stream tokens/steps and support background jobs.

The build is split into phases. Each phase has a clear deliverable and acceptance criteria so it can be handed off and verified independently.

Architecture (target)



Recommended stack - **API:** FastAPI + Uvicorn. Wraps the agent in-process (same Python runtime), so no subprocess/CLI parsing overhead. - **Streaming:** Server-Sent Events (SSE) for agent output. (WebSockets only if you need bidirectional mid-run input.) - **Background execution:** Celery or RQ with Redis. A run is started via the API, executed by a worker, and its events are streamed back through Redis pub/sub → SSE. - **DB:** PostgreSQL + SQLAlchemy + Alembic migrations. - **Auth:** Google OAuth 2.0 (Authorization Code flow) via Authlib. Session as an HttpOnly cookie holding a signed JWT. - **Frontend:** Next.js (React) + Tailwind. Use the Vercel AI SDK or a plain `EventSource` /fetch-stream reader for SSE. - **Deploy:** Docker Compose (api, worker, redis, postgres, frontend) → any VM or container host. Add a reverse proxy (Caddy/Nginx) for TLS.

Data model (minimum)

```
users
  id (uuid, pk)
  email (unique)           -- from Google
  name, avatar_url
  role                     -- 'admin' | 'member'
  is_active (bool)
  created_at, last_login_at

threads                    -- a conversation
  id (uuid, pk)
  user_id (fk -> users)
  title
  created_at, updated_at

messages
  id (uuid, pk)
  thread_id (fk -> threads)
  role                     -- 'user' | 'assistant' | 'tool'
  content (text/json)
  created_at

runs                      -- one agent execution
  id (uuid, pk)
  thread_id (fk)
  status                   -- 'queued' | 'running' | 'succeeded' | 'failed' | 'cancelled'
  error (nullable)
  started_at, finished_at
```

API contract (core endpoints)

Auth - `GET /auth/login` → redirect to Google consent - `GET /auth/callback` → exchange code, upsert user (reject non-company domains), set session cookie - `POST /auth/logout` → clear session - `GET /me` → current user + role

Threads & messages - `GET /threads` → list current user's threads - `POST /threads` → create thread - `GET /threads/{id}/messages` → message history - `DELETE /threads/{id}` → delete thread

Runs (streaming) - `POST /threads/{id}/runs` → enqueue a run with the user's message; returns `run_id` - `GET /runs/{id}/stream` → **SSE**: streams events `token`, `tool_call`, `step`, `done`, `error` - `POST /runs/{id}/cancel` → cancel an in-flight run

Admin (role = admin) - `GET /admin/users` → list users - `PATCH /admin/users/{id}` → set role / activate / deactivate

SSE event shape (example):

```
event: token
data: {"text": "Hello"}

event: tool_call
data: {"name": "search", "args": {...}}

event: done
data: {"run_id": "...", "status": "succeeded"}
```

Google SSO flow (domain-restricted)

1. Create OAuth credentials in Google Cloud Console → **Web application** client. Set redirect URI `https://<host>/auth/callback`.

2. Store `GOOGLE_CLIENT_ID` / `GOOGLE_CLIENT_SECRET` as env vars (never commit).
 3. On `/auth/callback` : verify the ID token, extract `email` + `hd` (hosted domain) or check the email suffix. **Reject any email not on the allowed company domain.**
 4. Upsert the user; first admin is seeded manually (env var `INITIAL_ADMIN_EMAIL`).
 5. Issue an HttpOnly, Secure, SameSite=Lax session cookie (signed JWT, ~7-day expiry). All API routes depend on an auth guard that loads the user and enforces role.
-

Phased milestones

Phase 0 — Project setup (0.5–1 day)

- Monorepo: `/api` , `/worker` , `/frontend` , `docker-compose.yml` , `.env.example` .
- Postgres + Redis running via Compose; Alembic initialized.
- **Done when:** `docker compose up` starts all services; `GET /health` returns 200.

Phase 1 — Agent wrapped as a service (1–2 days)

- Define a single `run_agent(thread_history, message) -> event stream` adapter around the existing agent. It should **yield** events (tokens/tool calls) rather than print to stdout.
- Worker executes runs and publishes events to Redis; API exposes SSE.
- **Done when:** a hardcoded request produces a streamed response end-to-end (no auth yet).

Phase 2 — Auth + user management (2–3 days)

- Google OAuth login/callback, domain restriction, session cookie, `/me` , logout.
- User table, roles, auth guard, admin endpoints.
- **Done when:** a company-domain user can log in; a non-domain user is rejected; admin can list/deactivate users.

Phase 3 — Persistence & threads (1–2 days)

- Threads/messages/runs persisted; history endpoints; runs linked to users.
- **Done when:** a user's conversations survive a page reload and are scoped to that user only.

Phase 4 — Frontend (3–4 days)

- Login page (Google button) → chat UI with streaming, thread sidebar, new/delete thread, cancel run.
- Admin page for user management (admin only).
- **Done when:** full flow works in the browser: log in → chat with streaming → revisit history.

Phase 5 — Hardening & deploy (1–2 days)

- Rate limiting, request size limits, structured logging, error tracking (Sentry).
- CORS locked to frontend origin; secrets via env; TLS via reverse proxy.
- Basic tests: auth guard, domain rejection, run lifecycle, one E2E happy path.
- **Done when:** deployed to the target host over HTTPS with real Google credentials.

Rough total: ~2–3 weeks for one engineer.

Non-functional requirements

- **Security:** HttpOnly/Secure cookies; domain allowlist; no secrets in repo; per-user data isolation enforced in every query.

- **Observability:** structured JSON logs with `run_id` / `user_id` ; error tracking; run status visible in DB.
- **Reliability:** runs survive API restart (they live in the worker/queue); cancellation works; failed runs recorded with error.
- **Cost/limits:** cap concurrent runs per user; timeout long runs.

Out of scope (call out for later)

- Billing / usage metering, multi-org tenancy, SSO providers beyond Google, mobile app.

Key risks / decisions to confirm with the builder

1. **Agent must be refactored to yield events** instead of printing — this is the main integration effort. Confirm the current agent's entry point can be adapted.
2. In-process vs subprocess execution — in-process is assumed (Python↔Python). If the CLI has heavy global state, isolate per-run.
3. Streaming transport: SSE (assumed) vs WebSocket — pick SSE unless mid-run user input is needed.